



Lab Exercise

Week 3: Nested While Loops and Pattern Generation

7th Grade Computer Science - Module 2



1 Lab Overview

This week you'll practice creating nested while loops to generate patterns and solve complex iteration problems. You'll work with logical operators to create sophisticated loop conditions and debug multi-level loop structures.

Time Allocation: 65% of session (approximately 60 minutes)

Prerequisites

Before starting, make sure you can:

- Create basic while loops with proper initialization, condition, and update
- Use **break** and **continue** statements effectively
- Write conditions using comparison operators (<, >, ==, etc.)
- Understand logical operators: **and**, **or**, **not**

1.1 Learning Objectives

By the end of this lab, you will be able to:

- Design and implement nested while loops for 2D pattern generation
- Apply logical operators to create complex loop conditions
- Debug nested loop structures using systematic tracing
- Create common patterns: triangles, rectangles, and number pyramids

2 Part 1: Basic Nested Loop Structure (*15 minutes*)

2.1 Exercise 1.1: Your First Rectangle

Create a program that draws a rectangle of asterisks using nested while loops.

Specifications:

- Outer loop: controls rows (3 rows)

- Inner loop: controls columns (5 columns per row)
- Output: 3×5 rectangle of asterisks

Starter Code:

```

1 # Complete this code to create a 3x5 rectangle
2 row = 0
3 while row < 3:
4     col = 0
5     # Add your inner loop here
6
7     print() # New line after each row
8     row = row + 1

```

Expected Output:

```

* * * * *
* * * * *
* * * * *

```

2.2 Exercise 1.2: Customizable Rectangle

Modify your code to accept user input for rectangle dimensions.

Requirements:

- Ask user for number of rows
- Ask user for number of columns
- Draw the rectangle with those dimensions
- Add input validation (must be positive numbers)

Test Cases:

- 2 rows, 4 columns → 2×4 rectangle
- 1 row, 6 columns → single line of 6 asterisks
- 4 rows, 1 column → vertical line of 4 asterisks

3 Part 2: Pattern Generation (20 minutes)

3.1 Exercise 2.1: Right Triangle Numbers

Create a right triangle pattern where each row shows numbers from 1 up to the row number.

Expected Output:

```

1
1 2
1 2 3
1 2 3 4

```

Tip

Hints:

- The inner loop's limit depends on the outer loop's current value
- Use `while col <= row:` for the inner condition
- Reset `col = 1` at the start of each row

3.2 Exercise 2.2: Left-Aligned Triangle

Create a left-aligned triangle using asterisks where row `n` has `n` asterisks.

Expected Output:

```
*
**
***
****
*****
```

Quick Check

Challenge: How would you create a right-aligned triangle?

3.3 Exercise 2.3: Inverted Triangle

Create an inverted triangle where the first row has the most asterisks and each subsequent row has one fewer.

For a 5-row inverted triangle:

```
*****
****
***
**
*
```

Rule

Key Insight: The inner loop condition should be `col < (total_rows - current_row)`

3.4 Exercise 2.4: Multiplication Table

Create a multiplication table using nested loops.

Requirements:

- Create a 5×5 multiplication table
- Format numbers so they align properly (use `f"{product:3}"` for 3-character width)
- Include row and column headers

Expected Output:

```

    1   2   3   4   5
1  1   2   3   4   5
2  2   4   6   8  10
3  3   6   9  12  15
4  4   8  12  16  20
5  5  10  15  20  25

```

4 Part 3: Complex Conditions and Logical Operators (15 minutes)

4.1 Exercise 3.1: Boundary-Safe Movement

Create a program that simulates moving around a 5×5 grid with boundary checking.

Requirements:

- Start at position (2, 2)
- Accept movement commands: w (up), s (down), a (left), d (right), q (quit)
- Use logical operators to prevent moving outside the 0-4 range for both x and y
- Display current position after each valid move
- Show a warning for invalid moves

Starter Code:

```

1 x, y = 2, 2 # Starting position
2 print(f"Starting position: ({x}, {y})")
3 print("Use w/a/s/d to move, q to quit")
4
5 while True:
6     move = input("Move: ").lower()
7
8     if move == "q":
9         break
10    elif move == "w" and y > 0: # Move up
11        # Complete this
12    elif move == "s" and y < 4: # Move down
13        # Complete this
14    # Add cases for 'a' and 'd'
15    else:
16        print("Invalid move or boundary reached!")
17
18    print(f"Current position: ({x}, {y})")

```

4.2 Exercise 3.2: Pattern with Conditions

Create a checkerboard pattern using nested loops and logical operators.

Requirements:

- 8×8 grid
- Use 'X' and 'O' to create a checkerboard pattern

- Position (row, col) shows 'X' if (row + col) is even, 'O' if odd

Expected Output (first few rows):

```
X O X O X O X O
O X O X O X O X
X O X O X O X O
...
```

5 Part 4: Debugging Practice (10 minutes)

5.1 Exercise 4.1: Find the Bug

Each of the following code snippets has a bug. Identify and fix them:

Bug 1: Infinite Loop

```
1 row = 0
2 while row < 3:
3     col = 0
4     while col < 3:
5         print("*", end=" ")
6         # What's missing here?
7     print()
8     row = row + 1
```

Bug 2: Wrong Variable

```
1 i = 0
2 while i < 3:
3     j = 0
4     while i < 3: # Spot the error!
5         print("*", end=" ")
6         j = j + 1
7     print()
8     i = i + 1
```

Bug 3: Logic Error

```
1 # Trying to create a triangle, but something's wrong
2 row = 0
3 while row < 4:
4     col = 0
5     while col <= row:
6         print(row, end=" ")
7         col = col + 1
8     print()
9     # Missing something?
```

5.2 Exercise 4.2: Trace Table Practice

Create a trace table for this nested loop:

```
1 outer = 1
2 while outer <= 2:
3     inner = 1
4     while inner <= 3:
```

```

5     print(f"({outer},{inner})", end=" ")
6     inner = inner + 1
7     print()
8     outer = outer + 1

```

Create a table with columns: Step — Loop — outer — inner — Condition Check — Output

6 Part 5: Challenge Problems (5 minutes)

6.1 Challenge 1: Diamond Pattern

Create a diamond pattern using asterisks. For a diamond with max width 5:

```

*
***
*****
***
*

```

Tip

Hint: Think of it as two triangles - one expanding, one contracting.

6.2 Challenge 2: Number Pyramid

Create a pyramid where each row contains the row number repeated row-number times:

```

1
22
333
4444
55555

```

6.3 Challenge 3: Pascal's Triangle (Advanced)

Research Pascal's Triangle and try to create the first 5 rows using nested loops:

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

```

7 Reflection Questions

1. **Pattern Recognition:** What similarities did you notice between creating different triangle patterns? How did changing the inner loop condition affect the output?
2. **Debugging Strategy:** When you encountered infinite loops or unexpected output, what steps did you take to identify the problem?

3. **Logical Operators:** How did using **and** and **or** in loop conditions help you solve the boundary-checking problems?
4. **Real-World Applications:** Think of three real-world scenarios where nested loops might be useful (hint: consider grids, schedules, or data processing).
5. **Complexity:** How does adding a second loop affect the total number of operations your program performs?

8 Submission Checklist

Before submitting your lab work, ensure you have:

- ☐ Completed all exercises in Parts 1-4
- ☐ Fixed all bugs in Exercise 4.1
- ☐ Created a complete trace table for Exercise 4.2
- ☐ Attempted at least one challenge problem
- ☐ Added comments to your code explaining the purpose of each loop
- ☐ Tested your programs with different inputs
- ☐ Answered the reflection questions

★ Landmark Moment

Next Week Preview: Next week we'll dive into strings! You'll learn how Python stores and manipulates text, why strings behave differently from numbers, and discover the concept of immutability through memory diagrams.